

Semana 6 - Limpieza SSIS

Proyecto: Producción agrícola de Colombia (EVA y FAOSTAT) – Etapa 3: Tratamiento ETL con SSIS

Andrés F. Rodríguez

Jonathan D. Chavarro

Grupo Wololo

Facultad de Ingeniería, Ingeniería de sistemas y computación

Universidad de Cundinamarca, Chía, Cundinamarca

Minería de Datos – 901N

Docente Edison Cañon

27 de septiembre de 2026

Contenido

1.	Introducción.....	4
1.1.	Conceptos clave que aplicamos.....	4
2.	Diagnóstico inicial y reglas de tratamiento.....	4
2.1.	Los conjuntos de datos.....	4
2.2.	Problemas de calidad que encontramos en la etapa 2.....	5
2.3.	Reglas de tratamiento para EVA.....	6
2.4.	Reglas de tratamiento para FAOSTAT.....	7
3.	Arquitectura de la solución ETL.....	7
3.1.	Zona de staging y trazabilidad.....	7
3.2.	Parámetros, variables y conexiones.....	8
4.	Paquete EVA: Control Flow y Data Flow.....	8
4.1.	Control Flow de EVA.....	8
4.2.	Apertura del lote y cálculo de límites IQR.....	9
4.3.	DFT_Extraccion_EVA: carga a staging.....	10
4.4.	DFT_Limpieza_EVA: transformación, validación y separación.....	11
5.	Paquete FAOSTAT: Control Flow y Data Flow.....	11
5.1.	Control Flow de FAOSTAT.....	11
5.2.	Límites IQR y unidad modal.....	12
5.3.	DFT_Extraccion_FAOSTAT: tres archivos en un solo staging.....	13
5.4.	DFT_Limpieza_FAOSTAT: transformación, validación y separación.....	13
6.	Configuración paso a paso de los componentes.....	14
6.1.	OLE DB Source y detección de duplicados.....	14
6.2.	Derived Column: limpieza de texto, formato y claves.....	15
6.3.	Data Conversion: tipificación con redirección de errores.....	16
6.4.	Lookup: validación contra catálogos.....	16
6.5.	Conditional Split: reglas de dominio y consistencia.....	16
6.6.	Lookups IQR y Derived Column de banderas.....	17
6.7.	Destinos OLE DB y archivo de errores.....	17
7.	Tratamiento de nulos, duplicados, formatos e inválidos.....	17
8.	Manejo de errores y registros enviados a revisión.....	18
8.1.	Event Handler OnError.....	18
8.2.	Rama de revisión.....	18
8.3.	Qué pasa con los registros en revisión.....	19
9.	Iteraciones del tratamiento.....	19
9.1.	Iteración 1: reglas base.....	20

9.2.	Iteración 2: dominio, consistencia y conversión final.....	20
9.3.	Iteración 3: atípicos, unidad modal y parametrización por lote.....	20
9.4.	Conteos por iteración y conciliación.....	21
10.	Comparación de resultados e indicadores de calidad.....	22
10.1.	Indicadores de calidad.....	22
10.2.	Ejemplos de datos antes y después.....	22
10.3.	Mejoras obtenidas y problemas pendientes.....	23
11.	Verificación de re-ejecución sin duplicados.....	24
12.	Repositorio, aplicación y video de demostración.....	24
12.1.	Contenido del repositorio.....	24
12.2.	Publicación en la aplicación Flask.....	25
12.3.	Guion del video de demostración.....	25
12.4.	Enlaces.....	25
13.	Conclusión.....	26



Semana 6 - Limpieza SSIS

1. Introducción

En la etapa 2 del proyecto hicimos el diagnóstico de calidad de nuestras dos fuentes de datos agrícolas de Colombia: las Evaluaciones Agropecuarias Municipales (EVA) y las estadísticas de la FAO (FAOSTAT). Ahí encontramos filas repetidas, números escritos en formatos distintos, celdas vacías, códigos que no existen en los catálogos oficiales, valores negativos, unidades de medida que no coinciden y algunos valores muy extremos. En esta etapa 3 el reto fue pasar del diagnóstico a la acción: construir un proceso ETL en SQL Server Integration Services (SSIS) que corrija lo que se puede corregir y separe lo que necesita revisión, sin perder ningún dato en el camino.

Para eso armamos la solución R3 con dos paquetes que comparten la misma base de datos (WoLoLo_ETL) y la misma lógica de control por lotes: EVA.dtsx, que procesa el CSV de las evaluaciones municipales, y FAOSTAT.dtsx, que procesa tres CSV de la FAO (QCL, QCL básicos y FS). Los dos siguen el mismo patrón: primero copian los datos tal cual a una tabla de staging, luego los limpian y validan, y al final los reparten en una tabla de **limpios** y una de **revisión**, donde se guarda el valor original y el motivo.

En este informe explicamos cómo lo hicimos paso a paso: las reglas que definimos, cómo quedaron el Control Flow y el Data Flow de cada paquete, cómo configuramos los componentes, qué pasó en las tres iteraciones y cómo mejoró la calidad de los datos. También contamos qué aprendimos y qué nos quedó pendiente.

1.1. Conceptos clave que aplicamos

Antes de entrar en los paquetes, repasamos algunos conceptos del curso que usamos todo el tiempo:

- **ETL (Extract, Transform, Load):** extraer los datos de una fuente, transformarlos (limpiarlos, validarlos, darles formato) y cargarlos en un destino listo para el análisis. En minería de datos es la base, porque un modelo entrenado con datos sucios da resultados poco confiables.
- **Staging:** una tabla intermedia donde guardamos los datos exactamente como llegaron. Nos sirve para no tener que volver a leer el archivo y para poder comparar siempre contra el original.
- **Trazabilidad:** poder seguir cualquier registro desde el resultado final hasta su fila en el archivo original. Lo logramos con IdStg y NumeroFila.
- **Rango intercuartílico (IQR):** la distancia entre el primer cuartil (Q1) y el tercero (Q3). Según la regla de Tukey, un valor es atípico si queda por debajo de $Q1 - 1,5 \cdot IQR$ o por encima de $Q3 + 1,5 \cdot IQR$.
- **Moda:** el valor que más se repite. La usamos en FAOSTAT para saber cuál es la unidad de medida "normal" de cada producto y elemento.
- **Idempotencia:** que ejecutar el mismo proceso varias veces con la misma entrada dé siempre el mismo resultado, sin duplicar registros.

2. Diagnóstico inicial y reglas de tratamiento

2.1. Los conjuntos de datos

EVA. El archivo eva_basicos_colombia.csv (UTF-8, separado por comas, con encabezados) tiene **48.932 registros y 18 columnas** con los códigos y nombres DANE de departamento y municipio, la

clasificación del cultivo, el año y el periodo, y las cuatro métricas de producción: área sembrada, área cosechada, producción y rendimiento.

Tabla 1

Columnas del archivo de origen EVA

#	Columna	Tipo en origen	#	Columna	Tipo en origen
1	CodigoDeptoDane	texto (50)	10	Periodo	texto (50)
2	Departamento	texto (255)	11	AreaSembrada	texto (50)
3	CodigoMunicipioDane	texto (50)	12	AreaCosechada	texto (50)
4	Municipio	texto (255)	13	Produccion	texto (50)
5	GrupoCultivo	texto (255)	14	Rendimiento	texto (50)
6	Subgrupo	texto (255)	15	CicloCultivo	texto (255)
7	Cultivo	texto (255)	16	EstadoFisico	texto (255)
8	DesagregacionCultivo	texto (255)	17	CodigoCultivo	texto (50)
9	Anio	texto (50)	18	NombreCientifico	texto (255)

FAOSTAT. Son tres archivos con la misma estructura de 12 columnas: `faostat_qcl_colombia.csv` (cultivos y ganadería, 9.274 filas), `faostat_qcl_basicos_colombia.csv` (productos básicos, 374 filas) y `faostat_fs_colombia.csv` (seguridad alimentaria, 1.138 filas), para un total de **10.786 registros**. Cada fila es un valor para un área, un elemento (por ejemplo, producción o área cosechada), un producto y un año, con su unidad.

Tabla 2

Columnas de los archivos FAOSTAT (QCL, QCL básicos y FS)

#	Columna	Contenido	#	Columna	Contenido
1	CodigoAmbito	Código del dominio FAO	7	CodigoProducto	Código del producto
2	Ambito	Nombre del dominio	8	Producto	Nombre del producto
3	CodigoArea	Código del país/área	9	CodigoAnio	Año o rango (p. ej., 20182020)
4	Area	Nombre del área	10	Anio	Año en texto
5	CodigoElemento	Código del elemento	11	Unidad	Unidad de medida
6	Elemento	Producción, área, etc.	12	Valor	Valor numérico (texto)

Nota. En los dos paquetes decidimos leer todas las columnas como texto. Si intentábamos convertir a número desde la extracción, cualquier valor raro hacía fallar la carga y perdíamos el dato original.

2.2. Problemas de calidad que encontramos en la etapa 2

Del perfilamiento de la etapa 2 salieron estos problemas. Los organizamos según la dimensión de calidad que vimos en clase:

Tabla 3

Problemas de calidad del diagnóstico de la etapa 2

Problema	Dataset	Dimensión	Lo que vimos en los datos	Registros afectados
Filas duplicadas	EVA y FAOSTAT	Unicidad	La misma clave de negocio repetida más de una vez	EVA: 0 FAOSTAT: 0
Números en formato colombiano	EVA	Conformidad de formato	Las 4 métricas escritas como texto con punto de miles y coma decimal ("1.499,40")	48.932 (100 %)
Celdas vacías o no numéricas	FAOSTAT	Compleitud / validez	280 valores vacíos en QCL y 2 valores "<0.1" en FS	282

Problema	Dataset	Dimensión	Lo que vimos en los datos	Registros afectados
Espacios de más y mayúsculas mezcladas	EVA y FAOSTAT	Conformidad de formato	Se revisaron todas las columnas de texto	0 (el control queda activo)
Municipios que no existen	EVA	Integridad referencial	1.098 códigos de municipio; todos con 5 dígitos y el prefijo de su departamento	0
Área cosechada mayor que la sembrada	EVA	Consistencia	AreaCosechada > AreaSembrada en el mismo registro	6.700 (13,69 %)
Valores negativos	EVA y FAOSTAT	Validez de dominio	Áreas, producción, rendimiento o valor < 0	0
Unidades distintas para lo mismo	FAOSTAT	Consistencia	Más de una unidad para el mismo dataset, producto y elemento	0
Años como rango	FAOSTAT	Conformidad de formato	CodigoAnio de 8 dígitos (p. ej., 20002002 = 2000-2002)	611 (FS)
Valores atípicos	EVA y FAOSTAT	Plausibilidad	Valores fuera de $\pm 1,5 \cdot \text{IQR}$ de su cultivo o de su producto y elemento	EVA: 7.908 · FAOSTAT: 337

Nota. Conteos calculados sobre los CSV originales de `app/data/`. Coinciden con el diagnóstico de la etapa 2 (6.700 áreas incoherentes, 280 valores nulos en QCL y 327 + 7 + 3 atípicos en FAOSTAT) y son la línea base de la sección 10. En EVA, los atípicos se cuentan sobre las filas aceptadas en la iteración 3.

2.3. Reglas de tratamiento para EVA

Para cada problema definimos una regla con código propio, que también guardamos en la tabla `cat.Regla`. La idea que seguimos fue **no borrar ni cambiar datos a escondidas**: si la corrección es obvia y no cambia el significado del dato (como el formato de un número), la hacemos en el flujo; si hace falta que una persona decida, mandamos el registro a revisión con su valor original y el motivo.

Tabla 4

Reglas de tratamiento de EVA: campo afectado, acción y justificación

Código	Problema	Campo(s) afectado(s)	Qué hacemos	Por qué
EVA-DUP	Duplicados	CodigoMunicipioDane, CodigoCultivo, DesagregacionCultivo, Anio, Periodo	Dejamos la primera aparición y las demás van a revisión, indicando su posición en el grupo.	Si una evaluación aparece dos veces, la producción total queda inflada.
(corrección)	Formato de números	AreaSembrada, AreaCosechada, Produccion, Rendimiento	Quitamos espacios y el punto de miles, y cambiamos la coma decimal por punto antes de convertir.	El valor es el mismo; solo cambia cómo está escrito.
(corrección)	Espacios y mayúsculas	Códigos, Municipio, Cultivo, Periodo, Anio	TRIM en las claves y UPPER en Municipio y Periodo.	Sin esto, "Chía" y " CHÍA" parecen distintos y fallan los cruces.
EVA-CONV	Vacíos o no convertibles	Métricas y Anio	Si no se puede convertir a número, la fila va a revisión con los valores originales.	Inventar un valor (imputar) cambiaría las estadísticas sin que nadie lo sepa.
EVA-MUN	Municipio inexistente	CodigoMunicipioDane	Lo buscamos en <code>cat.Municipio (DIVIPOLA)</code> . Si no está, va a revisión; si está, usamos el nombre oficial.	Así todos los nombres quedan iguales a los del DANE.
EVA-DOM	Negativos	Las 4 métricas	Si alguna es < 0, va a revisión indicando cuál.	No existe un área o una producción negativa.
EVA-CONS	Área incoherente	AreaCosechada, AreaSembrada	Si la cosechada es mayor que la sembrada, va a revisión.	No se puede cosechar más de lo que se sembró.
(bandera)	Atípicos	Las 4 métricas, por cultivo	IQR por cultivo; $\text{FlagAtipico}^* = 1$ si el valor queda fuera del rango. El registro sí se acepta.	Un valor extremo no siempre es un error; puede ser un municipio muy productivo.
EVA-CONV	Texto demasiado largo	12 campos de texto	Si no cabe en la columna final, va a revisión.	Así el paquete no se cae ni corta el texto sin avisar.

2.4. Reglas de tratamiento para FAOSTAT

Tabla 5

Reglas de tratamiento de FAOSTAT: campo afectado, acción y justificación

Código	Problema	Campo(s) afectado(s)	Qué hacemos	Por qué
FAO-DUP	Duplicados	Dataset, CodigoArea, CodigoElemento, CodigoProducto, CodigoAnio, Unidad	Se queda la primera aparición (RankDup = 1); las demás van a revisión.	Un mismo dato repetido duplica la producción del año.
(corrección)	Espacios	Todas las columnas de texto	LTRIM/RTRIM desde la consulta de origen y NULLIF para convertir el valor vacío en nulo.	Los cruces con catálogos y la detección de duplicados dependen de claves limpias.
(corrección)	Año como rango	CodigoAnio	Si tiene 8 dígitos, se parte en AnioInicio y AnioFin; si no, ambos toman el mismo año.	Así se puede filtrar por año aunque el dato sea un promedio de varios años.
FAO-CNV	Valor no convertible	Valor	Si no se puede convertir a decimal, la fila va a revisión con el texto original.	No queremos inventar valores.
FAO-DOM	Valor negativo	Valor	Si el valor es < 0, va a revisión.	La producción, el área o el rendimiento no pueden ser negativos.
(bandera)	Unidad distinta a la habitual	Unidad	Calculamos la unidad modal por dataset, producto y elemento; FlagUnidadAtipica = 1 si la fila usa otra.	Comparar toneladas con hectogramos daría resultados absurdos; se marca para revisarlo en el análisis.
(bandera)	Atípicos	Valor, por producto y elemento	IQR por CodigoProducto CodigoElemento; FlagAtipicoValor = 1 si queda fuera del rango.	Se marca, no se elimina.
FAO-CNVF	Caracteres no representables	Códigos de ámbito, área, elemento, producto y año	Si un código no cabe o no se puede pasar a la página de códigos 1252, va a revisión.	Evita errores al insertar en la tabla final.

3. Arquitectura de la solución ETL

3.1. Zona de staging y trazabilidad

Creamos la base de datos `Wololo_ETL` con cuatro esquemas: `dbo` (datos), `etl` (control de lotes), `audit` (bitácora) y `cat` (catálogos). Cada paquete tiene su tabla de **staging** (`dbo.stg_EVA` y `dbo.stg_FAOSTAT`) donde se guardan los datos tal cual vienen del CSV, más el `IdLote`, la ruta del archivo y el número de fila. En FAOSTAT además se guarda la columna `Dataset` para saber de cuál de los tres archivos viene cada fila. Tener el staging nos ayudó en tres cosas:

- Podemos rastrear cualquier registro limpio o en revisión hasta su fila original con `IdStg` y `NumeroFila`.
- Podemos repetir la limpieza sin volver a leer los archivos y comparar las iteraciones sobre exactamente los mismos datos.
- Podemos calcular los indicadores de la línea base directamente sobre los datos crudos.

Tabla 6

Objetos de base de datos utilizados por los dos paquetes

Esquema	Objeto	Para qué sirve
etl	<code>usp_AbrirLote</code>	Crea el lote. Si la clave ya existe, borra lo que había cargado ese lote y lo vuelve a abrir. Devuelve el <code>IdLote</code> .
etl	<code>usp_NumerarStaging</code>	Le pone número de fila (<code>NumeroFila</code>) a los registros del lote.
etl	<code>usp_CerrarLote</code>	Cierra el lote, haya terminado bien o con error.

Esquema	Objeto	Para qué sirve
audit	usp_RegistrarEvento / Log	Guarda los eventos: LOTE_ABIERTO, LOTE_REABIERTO, LOTE_CERRADO y ERROR_PAQUETE.
dbo	stg_EVA / stg_FAOSTAT	Staging con los CSV crudos.
dbo	EVA_Limpios / FAOSTAT_Limpios	Registros aceptados, ya tipados y con banderas.
dbo	EVA_Revision / FAOSTAT_Revision	Registros a revisar: valor original, categoría, motivo, campo y regla.
cat	Municipio	Catálogo DIVIPOLA (solo EVA).
cat	LimitelQR	Límites IQR por lote, dataset, campo y grupo (cultivo en EVA; producto elemento en FAOSTAT).
cat	UnidadReferencia	Unidad modal por dataset, producto y elemento (solo FAOSTAT).
cat	Regla	Catálogo de reglas de calidad (familias EVA y FAOSTAT).

3.2. Parámetros, variables y conexiones

Los parámetros están en `Project.params` y los comparten los dos paquetes. Así, para correr otra iteración solo cambiamos sus valores, sin editar el paquete:

Tabla 7

Parámetros de proyecto

Parámetro	Paquete	Valor actual	Uso
pRutaCsv	EVA	ruta de eva_basicos_colombia.csv	Archivo de origen de EVA.
pLoteClave	EVA	EVA-I1	Clave del lote (EVA-I1, EVA-I2, EVA-I3).
pIteracion	EVA	1	Número de iteración.
pReglasVersion	EVA	v1	Versión de reglas.
pRutaCsvFaostatQCL / QCLBasicos / FS	FAOSTAT	rutas de los 3 CSV	Archivos de origen de FAOSTAT.
pLoteClaveFaostat	FAOSTAT	FAOSTAT-I1	Clave del lote (FAOSTAT-I1 a FAOSTAT-I3).
pIteracionFaostat	FAOSTAT	1	Número de iteración.
pReglasVersionFaostat	FAOSTAT	v1	Versión de reglas (se usa en LKP_Regla).

Dentro de cada paquete usamos las variables `vIdLote` (la devuelve `usp_AbrirLote` y filtra todo lo demás), `vDataset` y `vTipo` (REAL, para distinguir cargas reales de pruebas). En FAOSTAT, `vArchivoOrigen` es una expresión que une las tres rutas con " | ".

Para la base de datos usamos la conexión OLE DB localhost (proveedor MSOLEDBSQL, base `Wololo_ETL`) con **autenticación de Windows**, por eso los paquetes no tienen usuario ni contraseña. Para los archivos hay una conexión Flat File por CSV (`CM_CSV_EVA`, `CM_CSV_FAOSTAT_QCL`, `CM_CSV_FAOSTAT_QCLBASICOS` y `CM_CSV_FAOSTAT_FS`, todas UTF-8) y una más, `CM_CSV_Errores`, para el archivo de errores de FAOSTAT.

[Insertar captura: Connection Managers del proyecto y parámetros en `Project.params`]

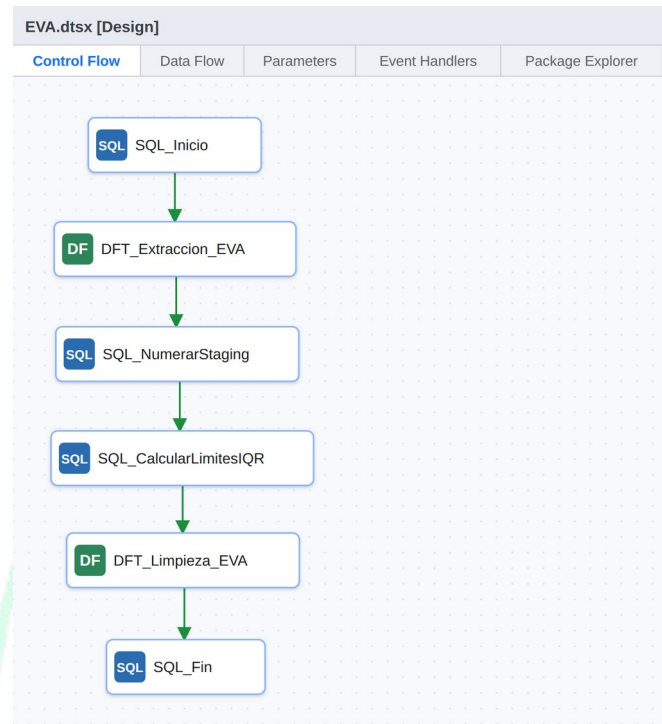
4. Paquete EVA: Control Flow y Data Flow

4.1. Control Flow de EVA

El Control Flow es el "orden de trabajo" del paquete. En EVA tiene seis tareas unidas con restricciones de tipo *Success* (flechas verdes): cada una arranca solo si la anterior terminó bien.

Figura 1

Control Flow del paquete EVA.dtsx



Nota. Diseño reconstruido a partir de las coordenadas del diseñador guardadas en el archivo EVA.dtsx.

Tabla 8

Tareas del Control Flow de EVA

Orden	Tarea	Tipo	Qué hace
1	SQL_Inicio	Execute SQL Task	Abre (o reabre) el lote con <code>etl.usp_AbrirLote</code> y guarda el <code>vIdLote</code> .
2	DFT_Extraccion_EVA	Data Flow Task	Copia el CSV a <code>dbo.stg_EVA</code> sin cambiar nada.
3	SQL_NumerarStaging	Execute SQL Task	Numera las filas del lote.
4	SQL_CalcularLimitesIQR	Execute SQL Task	Calcula Q1, Q3 y los límites IQR por cultivo en <code>cat.LimiteIQR</code> .
5	DFT_Limpieza_EVA	Data Flow Task	Limpia, valida, marca atípicos y separa aceptados de revisión.
6	SQL_Fin	Execute SQL Task	Cierra el lote con <code>etl.usp_CerrarLote</code> .

4.2. Apertura del lote y cálculo de límites IQR

SQL_Inicio ejecuta `{call etl.usp_AbrirLote(?, ?, ?, ?, ?, ?, ?, ?, ?, ?)}`. En *Parameter Mapping* enviamos nueve entradas (clave del lote, dataset, iteración, versión de reglas, tipo, ruta del CSV, nombre del paquete, nombre de la tarea y GUID de ejecución) y recibimos una salida: `User : vIdLote`. Si la clave del lote es nueva, se crea y se registra `LOTE_ABIERTO`. Si ya existía, el procedimiento borra todo lo que ese lote había cargado antes y registra `LOTE_REABIERTO`. Esta es la parte que hace que el proceso sea idempotente.

[Insertar captura: Editor de SQL_Inicio: pestaña Parameter Mapping con `vIdLote` como Output]

SQL_NumerarStaging llama a `etl.usp_NumerarStaging(?)` para numerar las filas en el orden en que se cargaron. Después, **SQL_CalcularLimitesIQR** calcula los cuartiles de cada métrica **por cultivo**,

porque no tiene sentido comparar el área de un cultivo de papa con la de uno de palma de aceite. Usamos PERCENTILE_CONT, que es la función de SQL Server para percentiles:

```

;WITH Base AS (
    SELECT LTRIM(RTRIM(Cultivo)) AS Cultivo,
           TRY_CONVERT(DECIMAL(14,2),
           REPLACE(REPLACE(LTRIM(RTRIM(AreaSembrada)),',',''),',','.')) AS AreaSembrada,
           ... -- AreaCosechada, Produccion, Rendimiento
    FROM dbo.stg_EVA WHERE IdLote = ?),
Cuartiles AS (
    SELECT DISTINCT Cultivo,
           PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY AreaSembrada) OVER (PARTITION BY Cultivo)
    AS Q1_AS,
           PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY AreaSembrada) OVER (PARTITION BY Cultivo)
    AS Q3_AS,
           ... FROM Base)
INSERT cat.LimiteIQR (IdLote, Dataset, Campo, GrupoClave, N, Q1, Q3, LimiteInferior,
LimiteSuperior)
SELECT ?, 'EVA', 'AreaSembrada', Cultivo, N, Q1_AS, Q3_AS,
       Q1_AS - 1.5*(Q3_AS - Q1_AS), Q3_AS + 1.5*(Q3_AS - Q1_AS) FROM Cuartiles
UNION ALL ... -- AreaCosechada, Produccion, Rendimiento

```

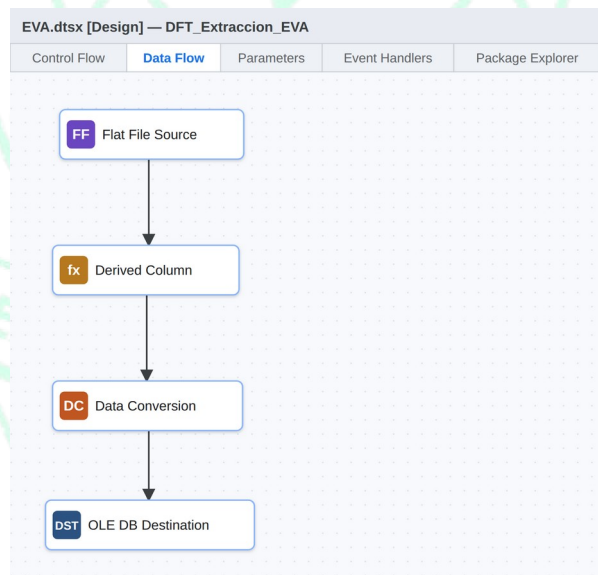
Los cinco ? se mapean a User::vIdLote. Como los límites se calculan con los datos del mismo lote que vamos a limpiar, se ajustan solos a cada ejecución. Al final, **SQL_Fin** ejecuta {call etl.usp_CerrarLote(?, DEFAULT, DEFAULT, DEFAULT, DEFAULT, ?, ?, ?)} y deja el lote como terminado.

4.3. DFT_Extraccion_EVA: carga a staging

Este primer Data Flow solo copia el archivo al staging. Aquí, a propósito, no aplicamos ninguna regla:

Figura 2

Data Flow DFT_Extraccion_EVA



Nota. Diseño reconstruido a partir de las coordenadas del diseñador guardadas en el archivo EVA.dtsx.

- **Flat File Source** (CM_CSV_EVA): lee las 18 columnas como DT_STR. Si hay un error, el componente falla, porque en esta parte no queremos perder ninguna fila.
- **Derived Column**: agrega IdLote = (DT_I4)@[User::vIdLote] y ArchivoOrigen = (DT_WSTR,260)@[Project::pRutaCsv].
- **Data Conversion**: pasa las 18 columnas a DT_WSTR (Unicode) para no dañar las tildes ni la ñ de los nombres.

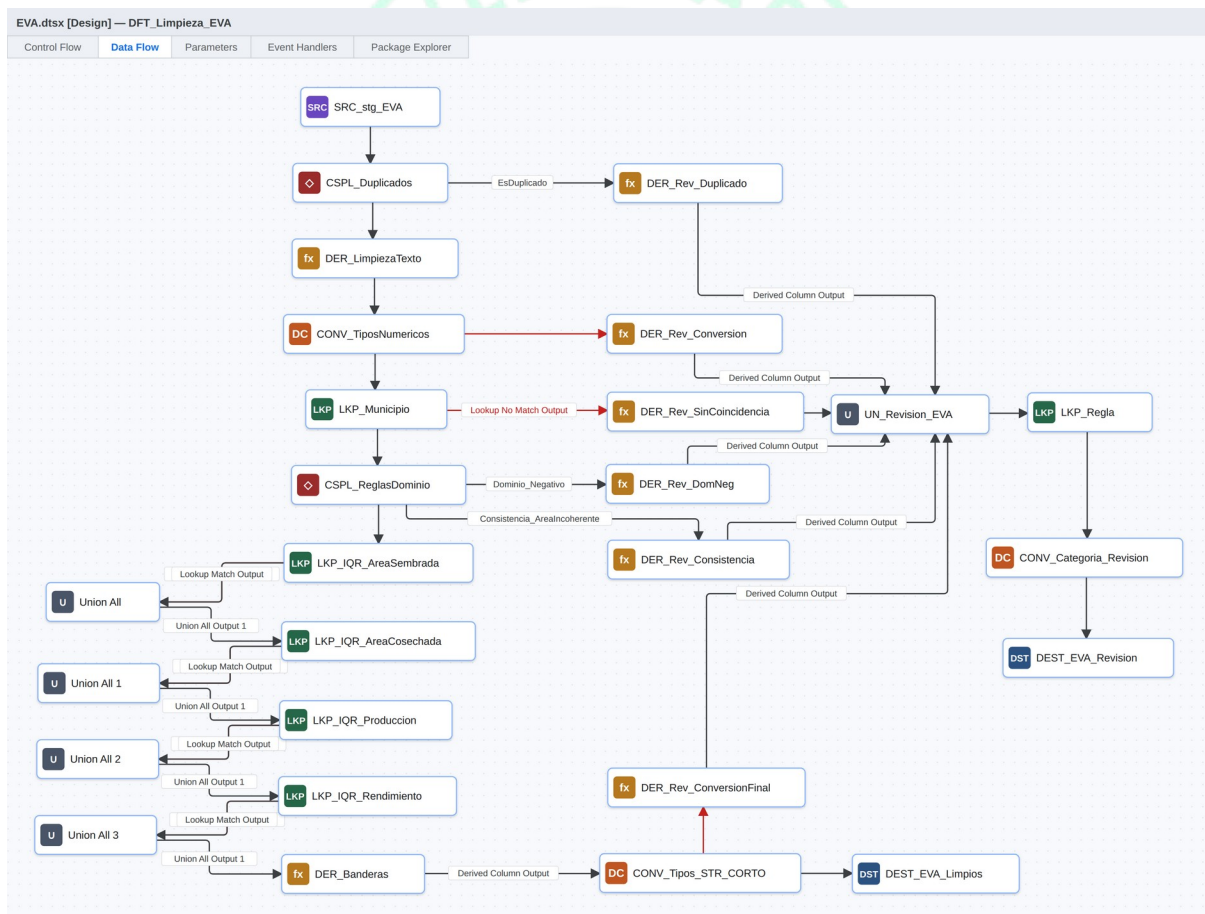
- **OLE DB Destination** (dbo.stg_EVA, Fast Load con TABLOCK, CHECK_CONSTRAINTS): carga las 18 columnas más IdLote y ArchivoOrigen. IdStg, NumeroFila y las fechas las genera la base de datos.

4.4. DFT_Limpieza_EVA: transformación, validación y separación

Este es el Data Flow más importante de EVA. Lee el staging del lote y lo divide en dos caminos: la **rama de limpios** (columna izquierda de la figura), que termina en DEST_EVA_Limpios, y la **rama de revisión** (derecha), que junta las seis salidas con problemas en UN_Revision_EVA y termina en DEST_EVA_Revision. Las flechas rojas son salidas de error o de "no coincidencia". Cada fila termina en uno solo de los dos destinos.

Figura 3

Data Flow DFT_Limpieza_EVA



Nota. Diseño reconstruido a partir de las coordenadas del diseñador guardadas en el archivo EVA.dtsx.

[Insertar captura: Data Flow DFT_Limpieza_EVA después de ejecutar, con los conteos de filas en cada ruta]

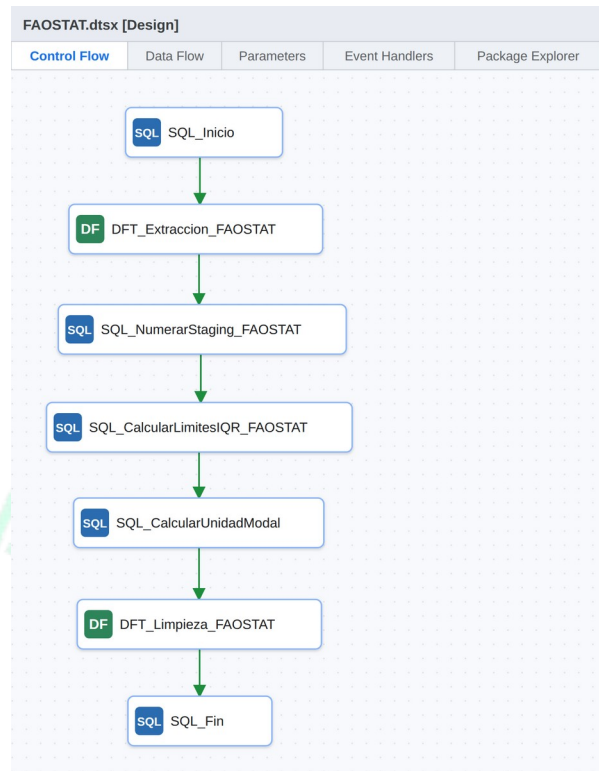
5. Paquete FAOSTAT: Control Flow y Data Flow

5.1. Control Flow de FAOSTAT

FAOSTAT sigue el mismo esquema que EVA, pero tiene una tarea más: SQL_CalcularUnidadModal, que prepara el catálogo de unidades antes de la limpieza.

Figura 4

Control Flow del paquete FAOSTAT.dtsx



Nota. Diseño reconstruido a partir de las coordenadas del diseñador guardadas en el archivo FAOSTAT.dtsx.

Tabla 9

Tareas del Control Flow de FAOSTAT

Orden	Tarea	Tipo	Qué hace
1	SQL_Inicio	Execute SQL Task	Abre (o reabre) el lote con etl.usp_AbrirLote y guarda el vIdLote.
2	DFT_Extraccion_FAOSTAT	Data Flow Task	Une los 3 CSV y los copia a dbo.stg_FAOSTAT.
3	SQL_NumerarStaging_FAOSTAT	Execute SQL Task	Numera las filas del lote.
4	SQL_CalcularLimitesIQR_FAOSTAT	Execute SQL Task	Calcula los límites IQR del valor por dataset y CódigoProducto CódigoElemento.
5	SQL_CalcularUnidadModal	Execute SQL Task	Guarda en cat.UnidadReferencia la unidad más frecuente de cada dataset, producto y elemento.
6	DFT_Limpieza_FAOSTAT	Data Flow Task	Limpia, valida, marca atípicos y separa aceptados de revisión.
7	SQL_Fin	Execute SQL Task	Cierra el lote con etl.usp_CerrarLote.

5.2. Límites IQR y unidad modal

En FAOSTAT solo hay una métrica (Valor), pero su escala cambia mucho según el producto y el elemento: no es lo mismo la producción de arroz en toneladas que el rendimiento del café. Por eso el IQR se calcula por la clave compuesta CódigoProducto|CódigoElemento dentro de cada dataset:

```
;WITH Base AS (  
    SELECT Dataset,  
           LTRIM(RTRIM(CódigoProducto)) + '|' + LTRIM(RTRIM(CódigoElemento)) AS GrupoClave,  
           TRY_CONVERT(DECIMAL(20,4), Valor) AS Valor  
    FROM dbo.stg_FAOSTAT WHERE IdLote = ?),
```

```

Cuartiles AS (
    SELECT DISTINCT Dataset, GrupoClave, COUNT(*) OVER (PARTITION BY Dataset, GrupoClave) AS
    N,
        PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY Valor) OVER (PARTITION BY Dataset,
        GrupoClave) AS Q1,
        PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY Valor) OVER (PARTITION BY Dataset,
        GrupoClave) AS Q3
    FROM Base WHERE Valor IS NOT NULL)
INSERT INTO cat.LimiteIQR (IdLote, Dataset, Campo, GrupoClave, N, Q1, Q3, LimiteInferior,
LimiteSuperior)
SELECT ?, Dataset, 'Valor', GrupoClave, N, Q1, Q3, Q1 - 1.5*(Q3 - Q1), Q3 + 1.5*(Q3 - Q1)
FROM Cuartiles;

```

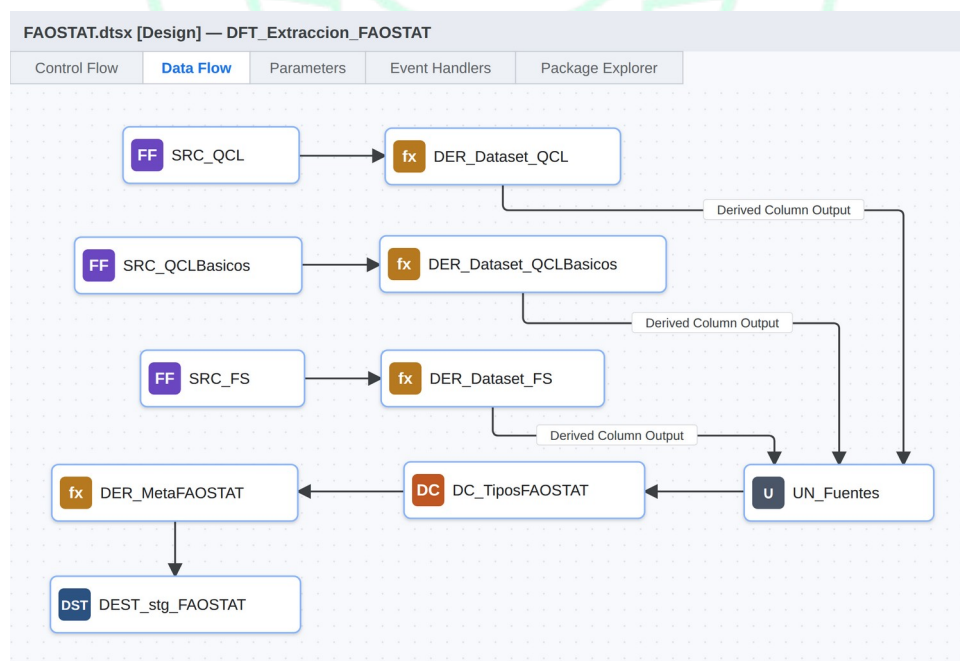
SQL_CalcularUnidadModal cuenta cuántas veces aparece cada unidad por dataset, producto y elemento, y se queda con la más frecuente con ROW_NUMBER() ... ORDER BY Frecuencia DESC, Unidad ASC (si hay empate, gana la primera en orden alfabético). El resultado va a cat.UnidadReferencia y lo usa después el Lookup LKP_UnidadReferencia.

[Insertar captura: Editor de SQL_CalcularUnidadModal y resultado de SELECT * FROM cat.UnidadReferencia]

5.3. DFT_Extraccion_FAOSTAT: tres archivos en un solo staging
 Aquí aprendimos a usar el **Union All** para juntar varias fuentes con la misma estructura. Cada archivo tiene su propio Flat File Source y su Derived Column, que agrega una columna Dataset con el nombre del origen ("QCL", "QCLBasicos" o "FS"). Después, UN_Fuentes junta las tres ramas, DC_TiposFAOSTAT pasa todo a Unicode (con sufijo _U), DER_MetaFAOSTAT agrega IdLote y ArchivoOrigen, y DEST_stg_FAOSTAT carga en dbo.stg_FAOSTAT.

Figura 5

Data Flow DFT_Extraccion_FAOSTAT



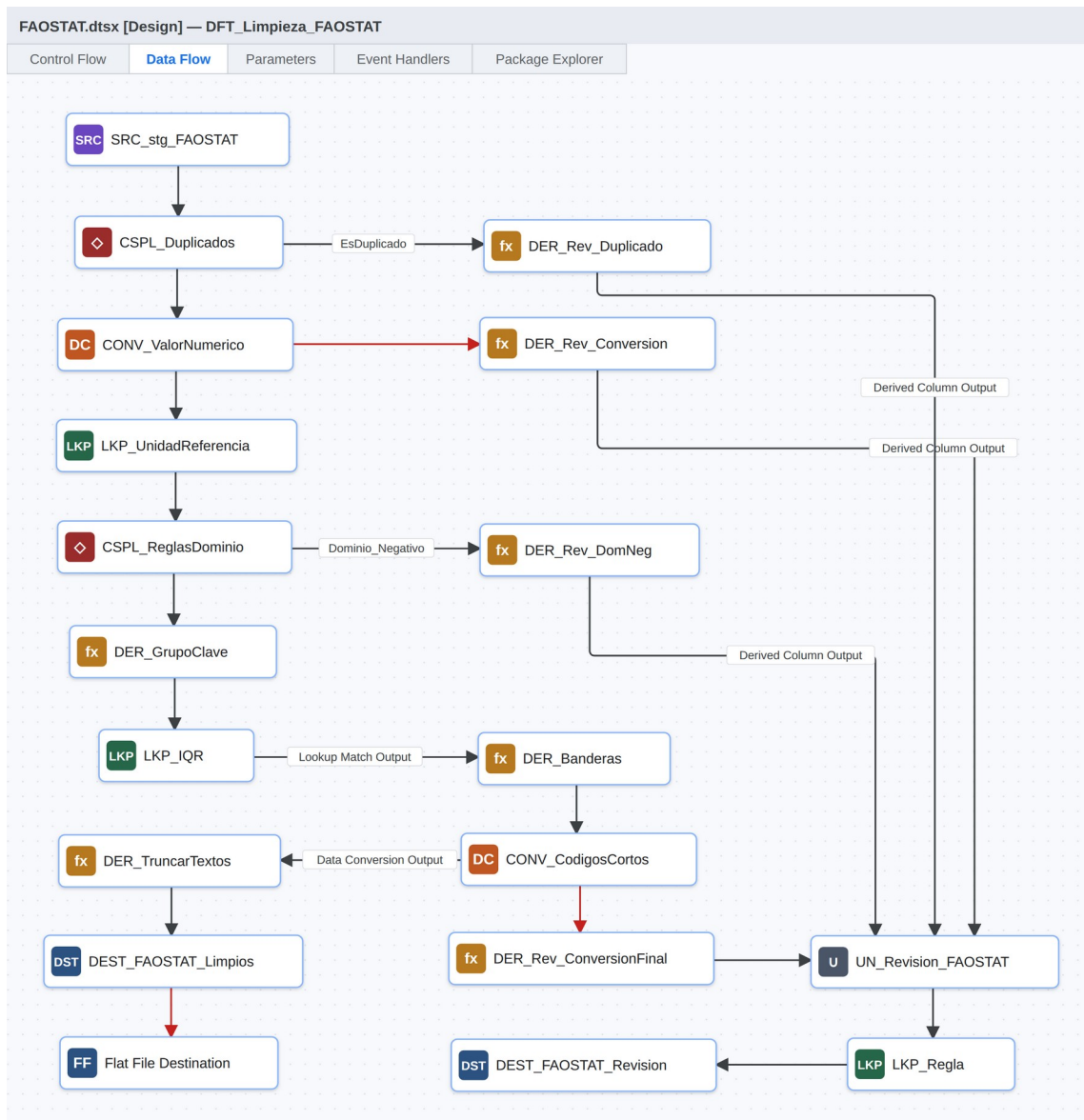
Nota. Diseño reconstruido a partir de las coordenadas del diseñador guardadas en el archivo FAOSTAT.dtsx.

5.4. DFT_Limpieza_FAOSTAT: transformación, validación y separación
 La lógica es parecida a la de EVA, pero con reglas propias de FAOSTAT: la conversión del valor, la unidad modal, el IQR por producto y elemento, y un recorte final de textos. Hay cuatro salidas hacia

la revisión y, además, una salida de error del destino de limpios que va a un archivo plano (errores_limpios.csv) para que ninguna fila que falle al insertarse se pierda.

Figura 6

Data Flow DFT_Limpieza_FAOSTAT



Nota. Diseño reconstruido a partir de las coordenadas del diseñador guardadas en el archivo FAOSTAT.dtsx.

[Insertar captura: Data Flow DFT_Limpieza_FAOSTAT después de ejecutar, con los conteos de filas en cada ruta]

6. Configuración paso a paso de los componentes

6.1. OLE DB Source y detección de duplicados

En los dos paquetes el origen del Data Flow de limpieza (**SRC_stg_EVA** y **SRC_stg_FAOSTAT**) es un OLE DB Source en modo *SQL command* con el parámetro ? mapeado a User : vIdLote. No leímos la tabla directamente, sino con una consulta que numera las repeticiones de cada clave. En EVA:

```
SELECT *,
```

```

COUNT(*) OVER (PARTITION BY LTRIM(RTRIM(CodigoMunicipioDane)),
LTRIM(RTRIM(CodigoCultivo)),
LTRIM(RTRIM(DesagregacionCultivo)), LTRIM(RTRIM(Anio)), LTRIM(RTRIM(Periodo))) AS
TotalEnGrupo,
ROW_NUMBER() OVER (PARTITION BY LTRIM(RTRIM(CodigoMunicipioDane)),
LTRIM(RTRIM(CodigoCultivo)),
LTRIM(RTRIM(DesagregacionCultivo)), LTRIM(RTRIM(Anio)), LTRIM(RTRIM(Periodo))
ORDER BY IdStg) AS OrdenEnGrupo
FROM dbo.stg_EVA
WHERE IdLote = ?

```

En FAOSTAT la consulta además limpia espacios con `LTRIM(RTRIM())`, convierte el valor vacío en nulo con `NULLIF(LTRIM(RTRIM(Valor)), '') AS ValorTexto`, separa el rango de años en `AnioInicio` y `AnioFin`, y calcula `RankDup` con `ROW_NUMBER()` sobre (`Dataset`, `CodigoArea`, `CodigoElemento`, `CodigoProducto`, `CodigoAnio`, `Unidad`).

Luego, **CSPL_Duplicados** (Conditional Split) manda a la salida `EsDuplicado` las filas repetidas (`TotalEnGrupo > 1 && OrdenEnGrupo > 1` en EVA; `RankDup > 1` en FAOSTAT). Las demás siguen por Continúa.

¿Por qué no usamos el componente Sort? SSIS permite quitar duplicados con Sort y la opción *Remove rows with duplicate sort values*, que es lo más común en los tutoriales. Pero Sort tiene que cargar todas las filas en memoria antes de dejar pasar una sola (es un componente bloqueante) y, además, borra los duplicados sin dejar rastro, lo que va en contra de la regla de guardar y justificar lo que separamos. Por eso preferimos que SQL Server hiciera el ordenamiento con `ROW_NUMBER() ... ORDER BY IdStg`.

[Insertar captura: Editor de SRC_stg_EVA (consulta y Parameters) y editor de CSPL_Duplicados]

6.2. Derived Column: limpieza de texto, formato y claves

En **DER_LimpiezaTexto** (EVA) creamos columnas nuevas en vez de reemplazar las originales. Así, si la fila termina en revisión, todavía tenemos el valor tal como llegó:

Tabla 10

Expresiones de DER_LimpiezaTexto (EVA)

Columna nueva	Tipo	Expresión	Para qué
CodigoMunicipioDaneClave	wstr(50)	TRIM(CodigoMunicipioDane)	Cruce con cat.Municipio
MunicipioClave	wstr(255)	UPPER(TRIM(Municipio))	Normalizar
CultivoClave	wstr(255)	TRIM(Cultivo)	Cruce con límites IQR
AnioClave	wstr(50)	TRIM(Anio)	Antes de pasarlo a entero
PeriodoClave	wstr(50)	UPPER(TRIM(Periodo))	Unificar el periodo
AreaSembradaTexto (y AreaCosechada, Produccion, Rendimiento)	wstr(50)	REPLACE(REPLACE(TRIM(AreaSembrada), ".", ""), ",", ".")	Quitar el punto de miles y cambiar la coma decimal
CodigoDeptoDaneClave, DesagregacionClave, CodigoCultivoClave	wstr(50/255)	TRIM(...)	Claves sin espacios

Con el doble REPLACE, "1.234,56" se vuelve "1234.56", que ya se puede convertir a decimal. El orden importa: si cambiáramos primero la coma por punto, después no habría forma de distinguir el punto de miles del decimal.

En FAOSTAT usamos tres Derived Column más: **DER_GrupoClave** arma `CodigoProducto + "|" + CodigoElemento` para cruzar con los límites IQR; **DER_Banderas** calcula las banderas (sección 6.6), y **DER_TruncarTextos** recorta con SUBSTRING los textos a la longitud de FAOSTAT_Limpios (Ambito, Area y Elemento a 100, Producto a 300, Anio a 20 y Unidad a 50).

[Insertar captura: Editor de DER_LimpiezaTexto con las expresiones]

6.3. Data Conversion: tipificación con redirección de errores

CONV_TiposNumericos (EVA) convierte las cuatro métricas a DT_NUMERIC con dos decimales y el año a DT_I2; **CONV_ValorNumerico** (FAOSTAT) convierte ValorTexto a DT_NUMERIC con cuatro decimales. La clave está en *Configure Error Output*: pusimos **Redirect row** en *Error* y en *Truncation*. Así, si una celda está vacía o tiene texto, el paquete no se detiene: la fila se va por la salida de error a DER_Rev_Conversion, que guarda el motivo.

Al final de la rama de limpios hay una segunda conversión: **CONV_Tipos_STR_CORTO** en EVA (12 campos a las longitudes de EVA_Limpios) y **CONV_CodigosCortos** en FAOSTAT (los cinco códigos a DT_STR con página de códigos 1252). También tienen **Redirect row**, y su salida de error va a DER_Rev_ConversionFinal.

[Insertar captura: Editor de CONV_TiposNumericos y ventana Configure Error Output con Redirect row]

6.4. Lookup: validación contra catálogos

LKP_Municipio (EVA) busca cada código de municipio en el catálogo oficial con caché completa: `SELECT CAST(CodigoMunicipioDane AS NVARCHAR(5)) AS CodigoMunicipioDane, CodigoDeptoDane, Departamento, Municipio FROM cat.Municipio`. El CAST es necesario porque el Lookup exige que las dos columnas del cruce sean del mismo tipo. Si encuentra el código, trae DepartamentoOficial y MunicipioOficial, que son los que guardamos en el destino. Si no lo encuentra, elegimos **Redirect rows to no match output** y la fila va a DER_Rev_SinCoincidencia.

LKP_UnidadReferencia (FAOSTAT) cruza Dataset, CodigoProducto y CodigoElemento con `cat.UnidadReferencia` y trae la UnidadModal. Su consulta se arma con una expresión de propiedad para que use el lote actual, y en *Configure Error Output* está en **Ignore failure**: si no hay coincidencia la fila sigue con UnidadModal nula y la bandera queda en 0.

[Insertar captura: Editor de LKP_Municipio: pestañas General, Connection y Columns]

6.5. Conditional Split: reglas de dominio y consistencia

CSPL_ReglasDominio revisa las condiciones en orden y manda cada fila por la primera que se cumpla:

Tabla 11

Condiciones de CSPL_ReglasDominio

Paquete	Salida	Condición
EVA	Dominio_Negativo	<code>AreaSembradaNum < 0 AreaCosechadaNum < 0 ProduccionNum < 0 RendimientoNum < 0</code>
EVA	Consistencia_ArealIncoherente	<code>AreaCosechadaNum > AreaSembradaNum</code>
FAOSTAT	Dominio_Negativo	<code>!ISNULL(ValorNumerico) && ValorNumerico < 0</code>

Paquete	Salida	Condición
Ambos	ContinuaValidacion (por defecto)	Todas las demás filas

[Insertar captura: Editor de CSPL_ReglasDominio]

6.6. Lookups IQR y Derived Column de banderas

En EVA usamos cuatro Lookups (LKP_IQR_AreaSembrada, LKP_IQR_AreaCosechada, LKP_IQR_Produccion, LKP_IQR_Rendimiento), uno por métrica, que traen de cat.LimiteIQR los límites del cultivo. En FAOSTAT hay uno solo, LKP_IQR, que cruza por Dataset y GrupoClave y también está en **Ignore failure**, porque hay 8 grupos producto|elemento (200 filas) que no tienen ningún valor numérico y por eso no tienen límites. En el diseño, la consulta de estos Lookups tiene un IdLote fijo; en ejecución se reemplaza por una expresión de propiedad sobre la variable del lote:

```
"SELECT GrupoClave, LimiteInferior, LimiteSuperior FROM cat.LimiteIQR
WHERE IdLote = " + (DT_WSTR,10)@[User::vIdLote] + "
AND Dataset = 'EVA' AND Campo = 'AreaSembrada'"
```

En EVA, si un cultivo no tiene límites, el Lookup lo manda por *No Match* y un **Union All** lo vuelve a juntar con la salida *Match* para no perder la fila. Después, **DER_Banderas** calcula para cada métrica:

```
(ISNULL(LimInf_AS) || ISNULL(LimSup_AS)) ? 0 : (AreaSembradaNum < LimInf_AS ||
AreaSembradaNum > LimSup_AS) ? 1 : 0
```

En palabras: si no hay límites, la bandera queda en 0; si los hay, vale 1 cuando el valor se sale del rango. En FAOSTAT, **DER_Banderas** calcula dos banderas: FlagAtipicoValor con la misma lógica, y FlagUnidadAtipica = ISNULL(UnidadModal) ? 0 : (Unidad != UnidadModal ? 1 : 0).

[Insertar captura: Editor de LKP_IQR_AreaSembrada, expresión de propiedad [SqlCommand] y editor de DER_Banderas]

6.7. Destinos OLE DB y archivo de errores

Los destinos (DEST_EVA_Limpios, DEST_EVA_Revision, DEST_FAOSTAT_Limpios y DEST_FAOSTAT_Revision) usan *Table or view – fast load* con TABLOCK, CHECK_CONSTRAINTS. En las tablas de limpios mapeamos los valores ya convertidos, los nombres oficiales, IdLote, IdStg y las banderas. En las de revisión, en cambio, mapeamos **los valores originales del staging**, sin tocar, junto con IdRegla, Categoria, Motivo, CampoAfectado y ValorOriginal.

En FAOSTAT, el destino de limpios tiene la salida de error en **Redirect row** hacia un **Flat File Destination** (CM_CSV_Errores → errores_limpios.csv), que guarda la fila completa con ErrorCode y ErrorColumn si algo falla al insertar.

[Insertar captura: Pestaña Mappings de DEST_EVA_Limpios y de DEST_EVA_Revision]

7. Tratamiento de nulos, duplicados, formatos e inválidos

Esta tabla resume cómo tratamos cada tipo de problema que pide la guía:

Tabla 12

Resumen del tratamiento por tipo de problema

Tipo de problema	Cómo lo tratamos en SSIS	Dónde termina el registro
Nulos o vacíos en métricas, año o valor	En EVA la celda llega vacía y la conversión falla; en FAOSTAT NULLIF la vuelve nula. En ambos casos la fila se redirige con su valor original si no se puede convertir.	Revisión (EVA-CONV / FAO-CNV)
Nulos en límites IQR o unidad modal	Si no hay referencia, la bandera queda en 0 y la fila sigue.	Limpios
Duplicados	ROW_NUMBER en el origen y Conditional Split; se queda la primera aparición.	La primera en limpios; el resto en revisión (EVA-DUP / FAO-DUP)
Formatos inconsistentes	TRIM, UPPER y doble REPLACE; nombres oficiales desde la DIVIPOLA; año en rango partido en inicio y fin.	Corregido en limpios
Valores inválidos	Negativos, áreas incoherentes, municipios inexistentes y códigos que no caben se separan.	Revisión (EVA-DOM, EVA-CONS, EVA-MUN, EVA-CONV, FAO-DOM, FAO-CNVF)
Valores atípicos y unidades distintas	IQR por cultivo o por producto elemento; unidad modal; se marcan pero no se eliminan.	Limpios con FlagAtipico* / FlagUnidadAtipica = 1

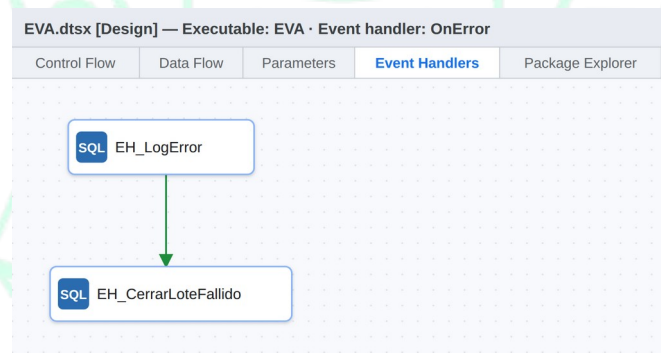
8. Manejo de errores y registros enviados a revisión

8.1. Event Handler OnError

Distinguimos dos tipos de error. Los **errores de datos** (una fila mala) no deben detener el proceso, y se manejan con las salidas de error y la rama de revisión. Los **errores del paquete** (se cae la conexión, falta el archivo) sí lo detienen, y para esos configuramos en los dos paquetes un Event Handler OnError con dos tareas:

Figura 7

Event Handler OnError del paquete EVA (FAOSTAT tiene el mismo diseño)



Nota. Diseño reconstruido a partir de las coordenadas del diseñador guardadas en el archivo EVA.dtsx.

- **EH_LogError** llama a `audit.usp_RegistrarEvento` con la acción `ERROR_PAQUETE`, la descripción del error (`System::ErrorDescription`), el `IdLote`, el componente que falló y el GUID de ejecución.
- **EH_CerrarLoteFallido** cierra el lote, pero solo si se cumple `!(ISNULL(@[User::vIdLote]) || @[User::vIdLote] == 0)`. Si el error pasó antes de abrir el lote, no tiene sentido intentar cerrarlo.

8.2. Rama de revisión

Cada salida con problemas pasa por un Derived Column `DER_Rev_*` que le agrega cinco columnas: `Categoria`, `Motivo`, `CampoAfectado`, `ValorOriginal` y `ReglaCodigo`. Luego se juntan en un

Union All (UN_Revision_EVA o UN_Revision_FAOSTAT), LKP_Regla busca el IdRegla en cat.Regla y se carga la tabla de revisión.

Tabla 13

Información que agrega cada rama de revisión

Componente	Categoría	Regla	Motivo que queda guardado
DER_Rev_Duplicado (EVA)	DUPLICADO	EVA-DUP	Fila duplicada en el lote: misma clave (posición X de N).
DER_Rev_Conversion (EVA)	ERROR_CONVERSION	EVA-CONV	Error de conversión numérica. ErrorCode / ErrorColumn.
DER_Rev_SinCoincidencia	SIN_COINCIDENCIA	EVA-MUN	CodigoMunicipioDane 'X' no existe en cat.Municipio (DIVIPOLA).
DER_Rev_DomNeg (EVA)	DOMINIO	EVA-DOM	Valor negativo en <campo>: <valor>.
DER_Rev_Consistencia	CONSISTENCIA	EVA-CONS	AreaCosechada (X) mayor que AreaSembrada (Y).
DER_Rev_ConversionFinal (EVA)	ERROR_CONVERSION	EVA-CONV	Error de conversión final de texto en alguno de los 12 campos.
DER_Rev_Duplicado (FAOSTAT)	DUPLICADO	FAO-DUP	Fila duplicada según la clave natural (posición X del grupo).
DER_Rev_Conversion (FAOSTAT)	ERROR_CONVERSION	FAO-CNV	El valor "X" no se pudo convertir a número decimal.
DER_Rev_DomNeg (FAOSTAT)	DOMINIO	FAO-DOM	El valor X es negativo; FAOSTAT_Limpios no admite valores negativos.
DER_Rev_ConversionFinal (FAOSTAT)	ERROR_CONVERSION	FAO-CNVF	Un código tenía un carácter que no se puede representar en la página de códigos 1252.

8.3. Qué pasa con los registros en revisión

En las tablas de revisión queda la fila completa tal como llegó, con su IdStg para encontrarla en el staging. Las columnas EstadoRevision (que empieza como pendiente), Resolucion, RevisadoPor y FechaRevision quedan para que alguien revise cada caso y anote qué decidió: corregirlo y volver a procesarlo, aceptarlo como válido o descartarlo explicando por qué. Con esta consulta vemos cuántos hay de cada tipo:

```
SELECT 'EVA' AS Paquete, Categoria, COUNT(*) AS Registros
FROM dbo.EVA_Revision WHERE IdLote = @IdLoteEva GROUP BY Categoria
UNION ALL
SELECT 'FAOSTAT', Categoria, COUNT(*)
FROM dbo.FAOSTAT_Revision WHERE IdLote = @IdLoteFao GROUP BY Categoria
ORDER BY Paquete, Registros DESC;
```

[Insertar captura: Resultado de la consulta por categoría en SSMS]

9. Iteraciones del tratamiento

Ejecutamos el tratamiento tres veces por paquete sobre los mismos archivos, cada una con su propia clave de lote (EVA-I1 a EVA-I3 y FAOSTAT-I1 a FAOSTAT-I3) y el parámetro pIteracion correspondiente. En cada iteración revisamos los resultados y con eso decidimos qué ajustar en la siguiente. En todas comprobamos que se cumpliera **Recibidos = Aceptados + Enviados a revisión**, para asegurarnos de que no se perdiera ninguna fila sin explicación.

Para tener un control independiente de SSIS escribimos el script scripts/etapa3_iteraciones.py, que aplica en Python exactamente las mismas reglas de los paquetes, iteración por iteración, sobre los CSV originales. Los conteos de esta sección salen de ese script y se contrastan con la consulta de conciliación en SSMS (sección 9.4). Los resultados quedan además en app/static/data/R3/iteraciones.json, que es lo que muestra la aplicación.

9.1. Iteración 1: reglas base

Qué aplicamos: carga a staging, limpieza de texto y formato numérico, duplicados (EVA-DUP, FAO-DUP), conversión numérica con redirección (EVA-CONV, FAO-CNV) y, en EVA, validación del municipio contra la DIVIPOLA (EVA-MUN).

Qué encontramos: en EVA pasaron las 48.932 filas: no hubo duplicados, todos los números se pudieron convertir después del doble REPLACE y los 1.098 códigos de municipio existen en el catálogo. Pero al revisar la tabla de limpios vimos que **6.700 filas (13,69 %) tenían más área cosechada que sembrada** y habían entrado como si fueran válidas, y que 1.327 filas tienen área sembrada igual a 0. En FAOSTAT se aceptaron 10.784 filas: solo 2 filas de FS se fueron a revisión porque el valor venía como texto (" <0.1 "), y las 280 filas de QCL sin valor pasaron con Valor nulo. En ninguno de los dos paquetes teníamos todavía forma de ver los valores extremos.

[Insertar captura: Ejecución de la iteración 1 de EVA y de FAOSTAT (Data Flow con conteos por ruta)]

9.2. Iteración 2: dominio, consistencia y conversión final

Qué ajustamos y por qué: como en la iteración 1 pasaron áreas imposibles, agregamos CSPL_ReglasDominio (EVA-DOM, EVA-CONS y FAO-DOM). También pusimos **Redirect row** en la conversión final (CONV_Tipos_STR_CORTO y CONV_CodigosCortos) con su rama DER_Rev_ConversionFinal, y en FAOSTAT agregamos DER_TruncarTextos y el archivo errores_limpios.csv, para que un texto largo no tumbara el paquete.

Qué encontramos: en EVA, las **6.700 filas incoherentes salieron a revisión** con la regla EVA-CONS y quedaron 42.232 aceptadas. Por cultivo, la mayoría son de maíz (3.156), yuca (1.610), frijol (759), papa (605) y arroz (570). No apareció ningún valor negativo ni ningún texto que no cupiera en su columna. En FAOSTAT las reglas nuevas no separaron ninguna fila más (0 negativos, 0 códigos problemáticos y 0 filas en errores_limpios.csv), lo que confirmó que los valores eran válidos; pero seguía sin haber forma de distinguir los valores extremos de los normales.

[Insertar captura: Ejecución de la iteración 2 de EVA y de FAOSTAT]

9.3. Iteración 3: atípicos, unidad modal y parametrización por lote

Qué ajustamos y por qué: para identificar los valores extremos agregamos el cálculo de límites IQR, los Lookups IQR y DER_Banderas; en FAOSTAT sumamos SQL_CalcularUnidadModal y LKP_UnidadReferencia para marcar unidades distintas. Además cambiamos el IdLote fijo de los Lookups por expresiones con @[User::vIdLote] y probamos volver a ejecutar el mismo lote (sección 11).

Qué encontramos: en EVA, **7.908 de las 42.232 filas aceptadas (18,73 %) quedaron con al menos una bandera:** 5.850 en área sembrada, 5.192 en área cosechada, 5.415 en producción y 1.511 en rendimiento. El maíz es el cultivo con más proporción de atípicos (20,7 %) y la papa el de menos (14,8 %). Nos llamó la atención que los límites inferiores salen negativos (por ejemplo, en maíz el rango de área sembrada es -185 a 335 ha), así que en la práctica solo se marcan los valores altos: la distribución es muy sesgada, con muchos municipios pequeños y pocos muy grandes. En FAOSTAT se marcaron **337 valores atípicos** (327 en QCL, 7 en QCL básicos y 3 en FS), los mismos que había detectado la etapa 2, y **0 unidades distintas a la modal**. Hay 8 grupos producto|elemento (200 filas) sin ningún valor numérico; como no tienen límites, su bandera queda en 0.

[Insertar captura: Ejecución de la iteración 3 de EVA y de FAOSTAT]

9.4. Conteos por iteración y conciliación

Tabla 14

Conteos de registros por iteración – EVA

Métrica	EVA-I1	EVA-I2	EVA-I3
Recibidos (stg_EVA)	48.932	48.932	48.932
Aceptados (EVA_Limpios)	48.932	42.232	42.232
Duplicados (EVA-DUP)	0	0	0
Error de conversión (EVA-CONV)	0	0	0
Sin coincidencia DIVIPOLA (EVA-MUN)	0	0	0
Dominio negativo (EVA-DOM)	no aplica	0	0
Área incoherente (EVA-CONS)	no aplica	6.700	6.700
Total enviados a revisión	0	6.700	6.700
Aceptados con ≥ 1 bandera de atípico	no aplica	no aplica	7.908
Diferencia sin explicar	0	0	0

Tabla 15

Conteos de registros por iteración – FAOSTAT

Métrica	FAOSTAT-I1	FAOSTAT-I2	FAOSTAT-I3
Recibidos (QCL 9.274 + QCLBasicos 374 + FS 1.138)	10.786	10.786	10.786
Aceptados (FAOSTAT_Limpios)	10.784	10.784	10.784
de ellos con Valor nulo	280	280	280
Duplicados (FAO-DUP)	0	0	0
Error de conversión (FAO-CNV)	2	2	2
Dominio negativo (FAO-DOM)	no aplica	0	0
Conversión final (FAO-CNVF)	no aplica	0	0
Filas en errores_limpios.csv	no aplica	0	0
Total enviados a revisión	2	2	2
Atípicos por valor / unidad distinta	no aplica	no aplica	337 / 0
Diferencia sin explicar	0	0	0

En SSMS la conciliación se revisa con esta consulta (en FAOSTAT es la misma cambiando las tablas; ambas están en etapa3/sql/04_consultas_verificacion.sql). La columna SinExplicar debe dar 0:

```
SELECT l.LoteClave,
       COUNT(*)
       SUM(CASE WHEN c.IdStg IS NOT NULL THEN 1 ELSE 0 END) AS Recibidos,
       SUM(CASE WHEN r.IdStg IS NOT NULL THEN 1 ELSE 0 END) AS Aceptados,
       SUM(CASE WHEN c.IdStg IS NULL AND r.IdStg IS NULL THEN 1 ELSE 0 END) AS Revision,
       SUM(CASE WHEN c.IdStg IS NULL AND r.IdStg IS NULL THEN 1 ELSE 0 END) AS SinExplicar
FROM dbo.stg_EVA s
JOIN etl.Lote l ON l.IdLote = s.IdLote
LEFT JOIN dbo.EVA_Limpios c ON c.IdStg = s.IdStg AND c.IdLote = s.IdLote
LEFT JOIN (SELECT DISTINCT IdLote, IdStg FROM dbo.EVA_Revision) r
ON r.IdStg = s.IdStg AND r.IdLote = s.IdLote
GROUP BY l.LoteClave ORDER BY l.LoteClave;
```

[Insertar captura: Resultado de la consulta de conciliación para los lotes EVA-I1 a EVA-I3 y FAOSTAT-I1 a FAOSTAT-I3 en SSMS]

10. Comparación de resultados e indicadores de calidad

10.1. Indicadores de calidad

Para medir la mejora usamos indicadores de las dimensiones de calidad que vimos en clase. La columna «Original» se calcula sobre los datos crudos del staging (N = 48.932 en EVA y 10.786 en FAOSTAT); las demás, sobre el conjunto aceptado en cada iteración.

Tabla 16

Indicadores de calidad de EVA: datos originales frente a cada iteración

Indicador	Fórmula	Original	I1	I2	I3
Unicidad	1 – duplicados / filas	100 %	100 %	100 %	100 %
Conformidad del formato numérico	métricas con punto decimal / filas	0 %	100 %	100 %	100 %
Integridad referencial	municipios en DIVIPOLA / filas	100 %	100 %	100 %	100 %
Validez de dominio	filas sin negativos / filas	100 %	100 %	100 %	100 %
Consistencia de áreas	filas con $AC \leq AS$ / filas	86,31 %	86,31 %	100 %	100 %
Atípicos identificados	filas con bandera / aceptados	sin medir	sin medir	sin medir	18,73 %
Tasa de aceptación	aceptados / recibidos	—	100 %	86,31 %	86,31 %

Tabla 17

Indicadores de calidad de FAOSTAT: datos originales frente a cada iteración

Indicador	Fórmula	Original	I1	I2	I3
Unicidad	1 – duplicados / filas	100 %	100 %	100 %	100 %
Compleitud del valor	filas con valor / filas	97,40 %	97,40 %	97,40 %	97,40 %
Validez de formato	valores convertibles / valores no vacíos	99,98 %	100 %	100 %	100 %
Validez de dominio	filas sin negativos / filas	100 %	100 %	100 %	100 %
Consistencia de unidades	filas con la unidad modal / aceptados	sin medir	sin medir	sin medir	100 %
Atípicos identificados	filas con bandera / aceptados	sin medir	sin medir	sin medir	3,12 %
Tasa de aceptación	aceptados / recibidos	—	99,98 %	99,98 %	99,98 %

Nota. La completitud de FAOSTAT no cambia porque decidimos no inventar los 280 valores que FAOSTAT no reportó: se aceptan con Valor nulo y quedan identificados con FlagValorNulo.

10.2. Ejemplos de datos antes y después

Tabla 18

Ejemplos reales: datos corregidos y registros enviados a revisión

Archivo y fila	Campo	Valor original	Resultado	Destino / motivo
EVA, fila 2 (Apartadó, arroz, 2019A)	Produccion	"1.499,40" (texto)	1499.40 (decimal)	EVA_Limpios: formato corregido
EVA, fila 2	Municipio	"Apartadó"	Nombre oficial de cat.Municipio	EVA_Limpios: homologado con DIVIPOLA
EVA, fila 9 (Apartadó 05045, arroz, 2022B)	AreaCosechada / AreaSembrada	"878,00" / "638,00"	Separado	EVA_Revision · EVA-CONS: AreaCosechada (878.00) mayor que AreaSembrada (638.00)
FS, fila 767 (niños < 5 años con emaciación, 2000)	Valor	"<0.1"	Separado	FAOSTAT_Revision · FAO-CNV: no se pudo convertir a número decimal

Archivo y fila	Campo	Valor original	Resultado	Destino / motivo
FS, fila 1 (suficiencia energética)	CodigoAnio	"20002002"	AnioInicio = 2000, AnioFin = 2002	FAOSTAT_Limpios: rango de años separado
QCL (avena)	Valor	"" (vacío)	NULL	FAOSTAT_Limpios con FlagValorNulo = 1

Nota. El número de fila es la posición dentro del CSV original (sin contar el encabezado). En la base de datos, el mismo registro se encuentra con NumeroFila en el staging.

Con esta consulta buscamos un mismo registro antes y después de la limpieza:

```
SELECT s.NumeroFila, s.Municipio, s.Produccion AS ProduccionOriginal, c.Produccion AS
ProduccionLimpia
FROM dbo.stg_EVA s
JOIN dbo.EVA_Limpios c ON c.IdStg = s.IdStg
WHERE s.IdLote = @IdLote AND s.NumeroFila = 2;
```

[Insertar captura: Resultado en SSMS del ejemplo corregido (fila 2) y del registro enviado a revisión (fila 9)]

10.3. Mejoras obtenidas y problemas pendientes

Lo que mejoró. Después de la tercera iteración, el conjunto aceptado de EVA no tiene duplicados, todas sus métricas son decimales (antes el 100 % venía con coma decimal), los nombres de municipios coinciden con el catálogo oficial y la consistencia de áreas pasó de 86,31 % a 100 %, porque las 6.700 filas incoherentes quedaron separadas con su valor original y su motivo. En FAOSTAT se separaron los 2 valores que venían como texto, los 611 años en rango quedaron partidos en inicio y fin, y los 337 atípicos quedaron marcados sin borrarlos. En ninguna iteración se perdió una fila sin explicación.

Lo que nos quedó pendiente. Al revisar los paquetes y los datos encontramos detalles que todavía hay que corregir:

- **FAOSTAT - longitud de `Producto`:** en QCL básicos la columna mide 50 caracteres y en QCL y FS 500, y el Union All UN_Fuentes tomó los 50 del primer archivo. Hay **1.650 filas** (533 de QCL y 1.117 de FS, 57 productos distintos) con nombres más largos que se recortan al pasar por el Union All. Se corrige en el *Advanced Editor* de UN_Fuentes poniendo la columna de salida Producto en 500. La consulta 8 de 04_consultas_verificacion.sql permite comprobarlo.
- **FAOSTAT - `LKP\_Regla`:** su salida *No Match* no está conectada. Hoy funciona porque cat.Regla tiene los cuatro códigos (FAO-DUP, FAO-CNV, FAO-DOM y FAO-CNVF), pero si faltara uno la fila se descartaría sin aviso. Conviene conectarla a un destino de error.
- **FAOSTAT - valores nulos:** las 280 filas sin valor entran a limpios; el paquete no llena FlagValorNulo (está en *ignore*), por eso en nuestro script SQL la dejamos como columna calculada.
- **FAOSTAT - `vDataset`:** el lote se abre con QCL aunque procesa los tres archivos.
- **EVA - `LKP\_Regla`:** la versión quedó fija (ReglasVersion = 'v1'); lo ideal sería tomarla de \$Project::pReglasVersion, como ya se hace en FAOSTAT.
- **Criterio de atípicos:** con distribuciones tan sesgadas la regla de 1,5·IQR marca casi 1 de cada 5 filas de EVA. Queremos probar calculando el IQR sobre el logaritmo de los valores o por cultivo y departamento.
- **Ambos:** falta gestionar la cola de revisión (EstadoRevision, Resolucion) y decidir qué hacer con las 6.700 filas de área incoherente.

11. Verificación de re-ejecución sin duplicados

Para probar que el proceso no duplica datos, ejecutamos cada paquete dos veces seguidas con la misma clave de lote (EVA-I3 y FA0STAT-I3). En la segunda ejecución, `etl.usp_AbrirLote` encuentra la clave, borra lo que ese lote había cargado (staging, limpios, revisión, límites IQR y unidades), aumenta `VecesAbierto` y registra `LOTE_REABIERTO`. Lo comprobamos con estas consultas:

```
-- 1. Conteos del lote (deben ser iguales en las dos ejecuciones)
SELECT (SELECT COUNT(*) FROM dbo.stg_EVA WHERE IdLote = @IdLote) AS Staging,
       (SELECT COUNT(*) FROM dbo.EVA_Limpios WHERE IdLote = @IdLote) AS Limpios,
       (SELECT COUNT(*) FROM dbo.EVA_Revision WHERE IdLote = @IdLote) AS Revision,
       (SELECT VecesAbierto FROM etl.Lote WHERE IdLote = @IdLote) AS VecesAbierto;

-- 2. Ninguna clave de negocio repetida en los datos limpios (debe devolver 0 filas)
SELECT CodigoMunicipioDane, CodigoCultivo, DesagregacionCultivo, Anio, Periodo, COUNT(*)
FROM dbo.EVA_Limpios WHERE IdLote = @IdLote
GROUP BY CodigoMunicipioDane, CodigoCultivo, DesagregacionCultivo, Anio, Periodo
HAVING COUNT(*) > 1;

-- 3. Bitácora: LOTE_ABIERTO, LOTE_CERRADO, LOTE_REABIERTO, LOTE_CERRADO
SELECT Fecha, Accion, Descripcion FROM audit.Log WHERE IdLote = @IdLote ORDER BY IdLog;
```

Tabla 19

Resultados de la verificación de idempotencia

Verificación	Resultado esperado	Ejecución 1	Ejecución 2 (mismo lote)
EVA-I3 – stg / limpios / revisión	48.932 / 42.232 / 6.700	[confirmar]	[confirmar]
FA0STAT-I3 – stg / limpios / revisión	10.786 / 10.784 / 2	[confirmar]	[confirmar]
Claves duplicadas en las tablas de limpios	0	[confirmar]	[confirmar]
VecesAbierto en etl.Lote	1 → 2	[confirmar]	[confirmar]
Eventos en audit.Log	LOTE_ABIERTO → LOTE_REABIERTO	[confirmar]	[confirmar]

[Insertar captura: Resultado de las tres consultas tras la segunda ejecución del mismo lote]

12. Repositorio, aplicación y video de demostración

12.1. Contenido del repositorio

Todo lo de esta etapa quedó en la rama `feature/etapa3` del repositorio, con lo necesario para reproducir el proceso **sin contraseñas ni credenciales**: la conexión usa autenticación de Windows y las rutas de los CSV se configuran como parámetros del proyecto en cada equipo.

```
Mineria-de-Datos/
├── app/
│   ├── data/
│   ├── routes/pages.py
│   ├── services/etapa3.py
│   ├── templates/pages/r3.html
│   └── static/
│       └── data/R3/iteraciones.json
├── etapa3/
│   └── ssis/
├── Project.params
├── sql/
├── verificación
│   ├── README.md
│   ├── scripts/etapa3_iteraciones.py
│   └── tests/test_etapa3.py
```

CSV originales (EVA y los 3 de FA0STAT)
nueva ruta /r3
contexto de la página de la etapa 3
sección Etapa 3
resultados de las 3 iteraciones
informe (PDF y Word) y diagramas
solución R3: R3.slnx, EVA.dtsx, FA0STAT.dtsx,
01 tablas · 02 procedimientos · 03 catálogos · 04
pasos para reproducir
réplica en Python de las reglas (control de conteos)

Para reproducir el proceso: se ejecutan en orden los scripts 01 a 03 en SQL Server, se abre etapa3/ssis/R3.slnx en Visual Studio con la extensión de SSIS, se ajustan los parámetros pRutaCsv* a la carpeta app/data del equipo y se ejecutan EVA.dtsx y FA0STAT.dtsx cambiando pLoteClave y pIteracion en cada iteración. El script 04 trae todas las consultas de evidencia.

12.2. Publicación en la aplicación Flask

En la aplicación agregamos la ruta /r3, **Etapá 3 – Tratamiento ETL con SSIS**, que aparece en el menú de etapas y en la hoja de ruta. Ahí se encuentra un resumen de todo el trabajo: las reglas de tratamiento, el diseño de los dos paquetes (con los diagramas), la tabla de las tres iteraciones con sus conteos y ajustes, la comparación de indicadores, los ejemplos antes y después, la prueba de re-ejecución, el video de demostración embebido y los botones para **descargar el informe en PDF y en Word**.

[Insertar captura: Sección Etapa 3 de la aplicación Flask publicada]

12.3. Guion del video de demostración

Tabla 20

Distribución del video (máximo 4 minutos, cámara encendida)

Tiempo	Integrante	Qué explica
0:00 – 1:00	Andrés F. Rodríguez	Diagnóstico, reglas de tratamiento y Control Flow de los dos paquetes (lotes, staging, IQR, unidad modal, OnError).
1:00 – 2:00	Jonathan D. Chavarro	Data Flow de limpieza: Derived Column, Data Conversion, Lookup, Conditional Split y rama de revisión.
2:00 – 3:00	Andrés F. Rodríguez	Ejecución de los paquetes; muestra el dato corregido de la fila 2 ("1.499,40" → 1499.40) y el registro de la fila 9 enviado a revisión por EVA-CONS.
3:00 – 4:00	Jonathan D. Chavarro	Resultados de las tres iteraciones, ajustes, indicadores, sección /r3 de la app y prueba de re-ejecución sin duplicados.

12.4. Enlaces

Recurso	URL
Repositorio de GitHub (rama feature/etapa3)	https://github.com/JODACHSE/Mineria-de-Datos/tree/feature/etapa3
Aplicación Flask publicada (sección Etapa 3)	https://wololo-seguridad-alimentaria.onrender.com/r3 — confirmar
Video de demostración	[URL del video]

13. Conclusión

Con esta etapa entendimos que limpiar datos no es solo "borrar lo que está mal". Los paquetes EVA.dtsx y FAOSTAT.dtsx corrigen lo que tiene una corrección clara, separan lo que necesita que alguien decida y marcan lo que es raro pero puede ser real, sin perder filas: de los 48.932 registros de EVA, 42.232 quedaron aceptados y 6.700 en revisión; de los 10.786 de FAOSTAT, 10.784 aceptados y 2 en revisión, y en los dos casos la diferencia sin explicar es cero. Gracias al staging y al control por lotes, además, podemos repetir el proceso las veces que queramos sin duplicar información.

Trabajar con dos fuentes distintas nos mostró que el mismo patrón sirve para problemas diferentes: en EVA el reto era el formato de los números y la coherencia entre áreas, y en FAOSTAT era juntar tres archivos, manejar años en rango y valores que venían como texto. Trabajar en tres iteraciones nos enseñó que la limpieza es un proceso de ensayo y ajuste: cada ejecución mostró algo que la anterior no veía. También nos quedó claro por qué en minería de datos se dice que la mayor parte del trabajo está en preparar los datos: sin esta etapa, cualquier modelo que construyamos después estaría aprendiendo de 6.700 registros imposibles. Como trabajo pendiente quedan los ajustes de la sección 10.3, sobre todo la longitud de Producto en FAOSTAT, y la gestión de la cola de revisión.



Bibliografía

- Kimball, R., y Caserta, J. (2004). *The data warehouse ETL toolkit: Practical techniques for extracting, cleaning, conforming, and delivering data*. Wiley.
- Departamento Administrativo Nacional de Estadística [DANE]. (s. f.). *División Político-Administrativa de Colombia – DIVIPOLA*. <https://www.dane.gov.co>
- Food and Agriculture Organization of the United Nations [FAO]. (s. f.). *FAOSTAT [Base de datos]*. <https://www.fao.org/faostat>
- Microsoft. (s. f.). *SQL Server Integration Services*. Microsoft Learn. <https://learn.microsoft.com/es-es/sql/integration-services/sql-server-integration-services>
- Microsoft. (s. f.). *Transformación Columna derivada*. Microsoft Learn. <https://learn.microsoft.com/es-es/sql/integration-services/data-flow/transformations/derived-column-transformation>
- Microsoft. (s. f.). *Transformación Conversión de datos*. Microsoft Learn. <https://learn.microsoft.com/es-es/sql/integration-services/data-flow/transformations/data-conversion-transformation>
- Microsoft. (s. f.). *Transformación Búsqueda*. Microsoft Learn. <https://learn.microsoft.com/es-es/sql/integration-services/data-flow/transformations/lookup-transformation>
- Microsoft. (s. f.). *Transformación División condicional*. Microsoft Learn. <https://learn.microsoft.com/es-es/sql/integration-services/data-flow/transformations/conditional-split-transformation>
- Microsoft. (s. f.). *Control de errores en los datos*. Microsoft Learn. <https://learn.microsoft.com/es-es/sql/integration-services/data-flow/error-handling-in-data>
- Ministerio de Agricultura y Desarrollo Rural. (s. f.). *Evaluaciones Agropecuarias Municipales – EVA [Conjunto de datos]*. Datos Abiertos Colombia. <https://www.datos.gov.co>
- Tukey, J. W. (1977). *Exploratory data analysis*. Addison-Wesley.